

Ejercicio 1

Clasifique las funciones y operadores miembro de la clase matriz en diferentes categorías: constructores (identificando sus diferentes tipos), destructores, observadores y modificadores. ¿Cuántas hay en cada categoría?

```
1 //Constructores
2 explicit matriz(size_t m = 1, size_t n = 1, double y = 0.0);
3 explicit matriz(size_t m = 1, size_t n = 1);
4 explicit matriz(size_t m = 1);
5 explicit matriz();
6 matriz(size_t m, size_t n, double f(size_t i, size_t j));
7 matriz(const initializer_list<valarray<double>>& l);
8 matriz(const matriz&);
9 matriz(matriz&&)
10
11 //Observadoras
12 size_t filas() const;
13 size_t columnas() const;
14 double operator()(size_t i, size_t j) const;
15 valarray<double> operator[](size_t i) const;
16 valarray<double> operator()(size_t j) const;
17
18 //Modificadoras
19 double& operator()(size_t i, size_t j);
20 slice_array<double> operator[](size_t i);
21 slice_array<double> operator()(size_t j);
22 matriz& operator=(const matriz&&);
23 matriz& operator=(matriz&&);
24 matriz& operator=(double y);
25 matriz& operator+=(const matriz& a);
26 matriz& operator-=(const matriz& a);
27 matriz& operator*=(const matriz& a);
28 matriz& operator*=(double y);
```

C++

Importante: No añadido las operaciones externas a la clase.

Ejercicio 2.1

Si existen los siguientes constructores, escriba en cada caso un ejemplo de instrucción en la que se invoque al mismo: Constructor predeterminado, Constructor de lista inicializadora, Constructor de copia, Constructor de movimiento.

```
1  int main()
2  {
3
4      matriz m0;
5      matriz m1{ {2.0, 2.1}, {4.5, 5.1} };
6      matriz m2{m0};
7      matriz m3 = matriz();
8
9      return 0;
10 }
```

C++

Ejercicio 2.2

Estudie el programa de prueba de las diapositivas de la clase matriz y diga si estos constructores se utilizan en el mismo y dónde.

```
1  int main()
2  {
3      matriz A(3, 3);          // Usa matriz(size_t m = 1, size_t n = 1, double y = 0.0),
                                // con y = 0.0
4      matriz B(3, 3, 2.0);    // Usa matriz(size_t m = 1, size_t n = 1, double y = 0.0),
                                // con y = 2.0
5      matriz C(3, 3, delta); // Usa matriz(size_t m, size_t n, double f(size_t i,
                                // size_t j))
6
7      matriz D = { {1, 2, 3}, // Usa matriz(initializer_list<valarray<double>> l)
                    {4, 5, 6},
                    {7, 8, 9} };
8
9
10
11     A = C; // Llama al operador de asignación por copia: matriz& operator=(const
matriz&)
12
13     cout << "A =\n" << A << endl;
14
15     B += -A;
16     // -A llama al operador externo friend matriz operator-(const matriz& a)
17     // Dentro de operator- se crea una copia: matriz c(a), por tanto se usa el
18     // constructor de copia
19     // Después B += ... llama a matriz& operator+=(const matriz& a)
20
21     cout << "B =\n" << B << endl;
22
23     C *= C += C;
24     // Primero se evalúa C += C, que llama a matriz& operator+=(const matriz& a)
25     // Después se evalúa C *= C, que llama a matriz& operator*=(const matriz& a)
26     // Dentro de operator*= se ejecuta: *this = *this * a
```

C++

```
26 // Eso llama al operador externo matriz operator*(const matriz& a, const matriz&
    b)
27 // El resultado temporal se asigna con operator=(matriz&&), es decir, asignación
    por movimiento
28
29 cout << "C =\n" << C << endl;
30
31 cout << "2A + B * C =\n"
32     << 2 * A + B * C << endl;
33 // 2 * A llama a operator*(double y, const matriz& a)
34 // Ese operador devuelve a * y, por tanto llama a operator*(const matriz& a,
    double y)
35 // En operator*(const matriz& a, double y) se crea una copia con matriz(a),
    usando constructor de copia
36 // B * C llama a operator*(const matriz& a, const matriz& b)
37 // 2 * A + B * C llama a operator+(const matriz& a, const matriz& b)
38 // operator+ crea una copia con matriz(a), usando constructor de copia
39 // y después llama a operator+=(const matriz& b)
40
41 cout << "D =\n" << D << endl;
42 }
```

Ejercicio 3

Describe los errores que hay en el siguiente código:

```
1 matriz A = 3; //Error: ya que el constructor que recibe un único size_t debe
  ser explícito.
2 matriz B = matriz(5); //Correcto
3 matriz C(3); //Correcto
4 B = 2; //Correcto llama al operador de asignación a partir de un double
5 A = matriz(5); //Si se solucionan errores anteriores correcto.
```

C++

¿Por qué se declara `explicit` el primer constructor de la clase `matriz`? **Se declara `explicit` para evitar conversiones implícitas no deseadas a partir de un número entero**

¿Podría causar algún problema si no se hiciera así? En tal caso, ¿se podría evitar ese problema definiendo un operador de conversión de `int` a `matriz`? Razone la respuesta.

Si no se indica podría existir problema de ambigüedad ya que la siguiente expresión `B = 2` se podría interpretar como crea una matriz temporal con 2 elementos y asignarlo a B.

No se resolvería definiendo un operador de conversión de `int` a `matriz` porque este operador se añade como miembro de la clase origen y no podemos modificar un tipo integral como lo es `int`. Además estaríamos ante el mismo problema de ambigüedad.

Ejercicio 4

¿Qué tipos devuelven los operadores de signo `+` y `-`? ¿Por qué devuelven tipos distintos?

`operator +` devuelve `const matriz&` porque el resultado es el mismo objeto. `operator -` devuelve `matriz` porque el resultado es una nueva matriz que es la opuesta de la que recibe como parámetro.

Ejercicio 5

¿El operador `-` de cambio de signo es miembro de la clase `matriz`?

El operador `-`, no es miembro de la clase pero es función amiga.

¿Se podría definir de la otra forma? En caso afirmativo, escriba la declaración. ¿Qué ventajas e inconvenientes tendría?

Se podría definir dentro, la ventaja es que al hacerla miembro no necesitaría ser `friend` para acceder a la parte interna mejorando la ocultación de información. La desventaja es que perdemos la simetría con la declaración del resto de operaciones. Cabecera: `matriz operator -() const`

Ejercicio 6

¿Es correcto definir el operador `*=` de matriz como sigue?

```
1 inline matriz& matriz::operator \*=(const matriz& a)
2 {
3     n = a.columnas();
4     x *= a.x;
5     return *this;
6 }
```

 C++

No es correcto porque `x *= a.x` multiplica los elementos componente a componente, mientras que la multiplicación de matrices requiere productos fila por columna. Además, modificar `n` no reconstruye correctamente la matriz resultante. Por eso debe calcularse primero `*this * a` y asignar después el resultado a `*this`.